Design Decision Document — Personalized Weather Station

1. Primary Sensor Selection — DHT22 to BME280

What we were trying to achieve: A reliable sensor capable of measuring temperature, humidity, and barometric pressure.

Options considered: DHT22, BME280

What we chose: BME280

Why: The DHT22 was used initially purely out of convenience — it was already on hand while the BME280 was on order. It served as a proof of concept and validated the sensing approach. Once the BME280 units arrived they were damaged during initial testing due to incorrect pin labeling on the component, causing improper connections. After resolving the pin issue and getting the BME280 operational it was the clear choice — it measures temperature, humidity, and barometric pressure in a single package versus the DHT22 which only handles temperature and humidity. The DHT22 was dropped entirely once the BME280 was confirmed working.

2. Communication Protocol — SPI Wiring, I2C Protocol

What we were trying to achieve: Reliable communication between the ATmega328P and BME280.

Options considered: SPI, I2C

What we chose: SPI wiring, I2C protocol

Why: The BME280 supports both SPI and I2C. The KiCad component library includes the BME280 footprint wired in SPI configuration, however the device operates over I2C. This approach leverages the existing KiCad symbol while maintaining I2C communication — the CSB pin is tied high to select I2C mode on the BME280. This gave us the benefits of I2C's simpler two-wire communication while working within the available schematic tooling.

3. Microcontroller Selection — ATmega328P

What we were trying to achieve: A capable, well-documented microcontroller that could handle sensor communication, data processing, and output while keeping the design lean and production-realistic.

Options considered: ATmega328P, full Arduino Uno board

What we chose: ATmega328P bare chip

Why: Rather than relying on a full development board, integrating the ATmega328P directly onto the PCB reflects how real embedded products are designed. It eliminates unnecessary overhead, reduces board size, and gives full control over the supporting circuitry. This approach results in a cleaner, more professional design that mirrors industry practice.

4. Crystal Oscillator vs Internal RC Oscillator

What we were trying to achieve: Stable and accurate clock signal for the MCU.

Options considered: Internal RC oscillator, external crystal oscillator

What we chose: External crystal oscillator (Y1) with 22pF load capacitors

Why: Professor recommendation to include an external crystal as good practice. The internal RC oscillator is less precise and more susceptible to temperature variation. An external crystal provides a more stable clock which is important for reliable SPI communication and timing accuracy.

5. PCB Layer Count — 2 Layer to 4 Layer Experimentation

What we were trying to achieve: A clean routable PCB layout with proper signal and power separation.

Options considered: 2-layer, 4-layer

What we chose: 2-layer as the primary design, 4-layer as an experiment

Why: The initial PCB was designed as a standard 2-layer board. During the design review it was noted that the routing crossed on both the front and back copper layers leaving limited room for additional traces. Rather than leaving that space unused, the opportunity was taken to experiment with a 4-layer stackup to learn via-to-layer assignment and inner copper layer routing. This was a learning exercise and the 2-layer version remains the primary submitted design.

6. Power Input — Dual Screw Terminals

What we were trying to achieve: Flexible power input that supports both wall power and battery operation.

Options considered: USB connector, barrel jack, screw terminals

What we chose: Two screw terminals (J1, J3)

Why: Having two power input options — wall adapter or battery — adds versatility with minimal added complexity. Screw terminals are easy to implement and allow any power source to be connected without needing a specific connector. USB was considered but did not offer a practical advantage for this application. The dual input approach means the board is never limited to a single power source.

7. Output Method — Touchscreen to Phone Notifications

What we were trying to achieve: A practical and user-friendly way to deliver sensor readings to the user.

Options considered: Touchscreen display with Raspberry Pi, phone notifications via push button

What we chose: Phone notifications triggered by push button

Why: The initial prototype used a Raspberry Pi and touchscreen combination. This was flagged as impractical for a real product — the Pi and screen add significant cost, size, and complexity. Phone notifications are far more versatile, require no dedicated display hardware, and fit how users actually interact with data today. The professor also recommended moving away from the Pi and touchscreen combo. While the notification approach adds some software complexity it results in a much more practical and scalable product.

8. Passive Component Values

What we were trying to achieve: Correct supporting circuitry for stable MCU and sensor operation.

Options considered: Various resistance and capacitance values per datasheet recommendations

What we chose: R1 10kΩ, R2/R3 4.7kΩ, R4 330Ω, C3/C4 22pF, C1/C2 and C5 decoupling caps

Why: Values were selected primarily from component datasheets. The 4.7kΩ pull-ups on the SPI communication lines ensure signal integrity. The 10kΩ reset pull-up keeps the MCU out of reset during normal operation. The 330Ω resistor limits current through the status LED to a safe level. The 22pF load capacitors are the standard recommended value for the crystal oscillator. Decoupling capacitors were added during design review to filter power supply noise on VCC, AVCC, and the BME280 VDDIO pin.